
boolean



React

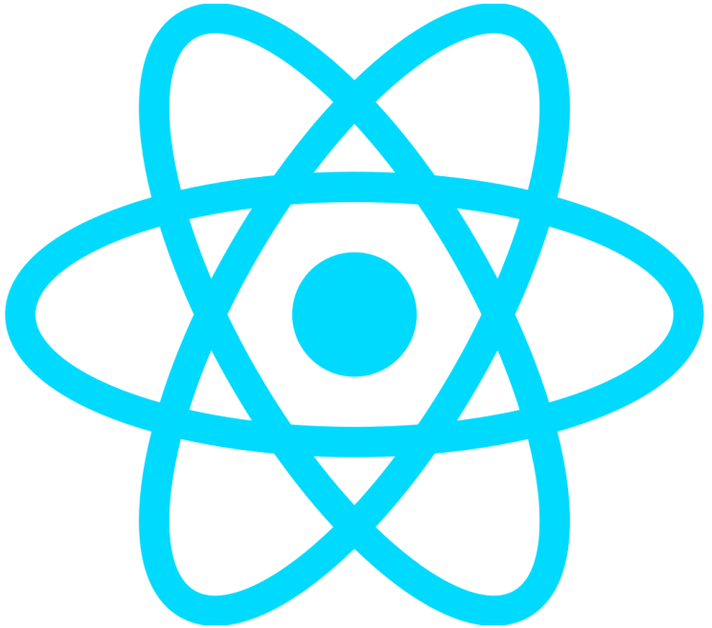
Intro

Intro

Perché esistono framework e librerie front-end come React?

Quando si sviluppa un'applicazione in JavaScript Vanilla, ci sono alcuni aspetti complessi da gestire:

- La **creazione** dinamica di elementi
- La **sincronizzazione** fra la **UI** e i **dati**
- Il salvataggio dei **dati**
- La **suddivisione** delle funzionalità per favorire l'organizzazione e la manutenibilità del progetto.



React ci aiuta a risolvere tutti questi problemi.

Intro

Che cos'è React?

React è una libreria JavaScript utilizzata per creare **interfacce utente**.

Ogni applicazione React è composta da **componenti riutilizzabili** che costituiscono porzioni dell'interfaccia utente e facilitano la strutturazione modulare dell'applicazione.

Ad esempio:
- barra di navigazione
- menu laterale
- contenuto principale
- footer





COMPONENTI

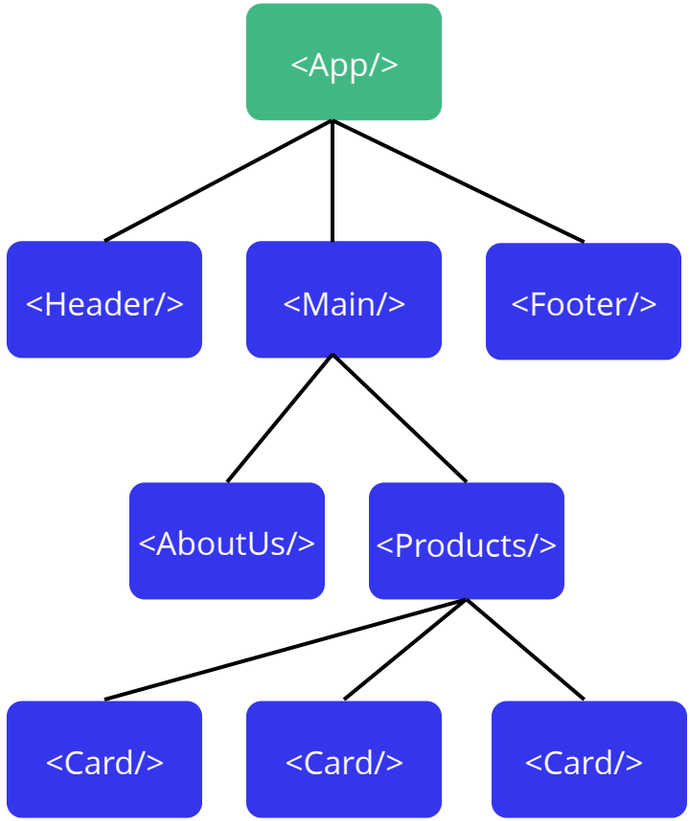
divide et impera

Componenti

I **Componenti** ci permettono di suddividere la UI in parti **indipendenti** e **riutilizzabili**, per poter pensare ad ognuna di esse in modo isolato.

App.jsx è il **componente radice** dell'applicazione, contiene la struttura di base e organizza gli altri componenti per la visualizzazione dell'applicazione.

Questo modulo viene importato e renderizzato dal file **main.jsx** nel documento index.html.



Componenti

Esportazione e importazione

Per creare un componente basta definire ed esportare una semplice funzione JavaScript all'interno di un modulo.
La funzione restituisce del markup **JSX**.

È una pratica comune utilizzare il PascalCase sia come nome del componente che come nome del file che lo contiene.

Quando vogliamo utilizzare un componente all'interno di un altro componente, dobbiamo prima **importarlo** e poi **inserirlo** come tag all'interno del **JSX** restituito.

```jsx
// Header.jsx
export default function Header() {
  return (<header>Sono l'header 😎</header>);
}
```

```jsx
// App.jsx
import Header from "./components/Header";

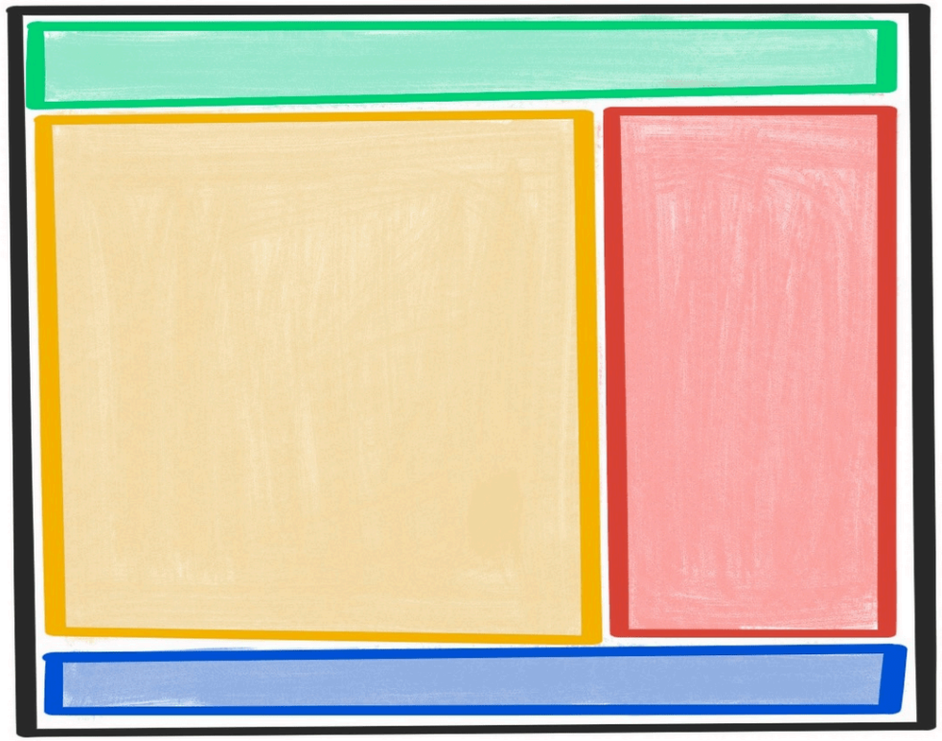
export default function App() {
  return (<Header />);
}
```

Componenti

Componenti per il macro Layout

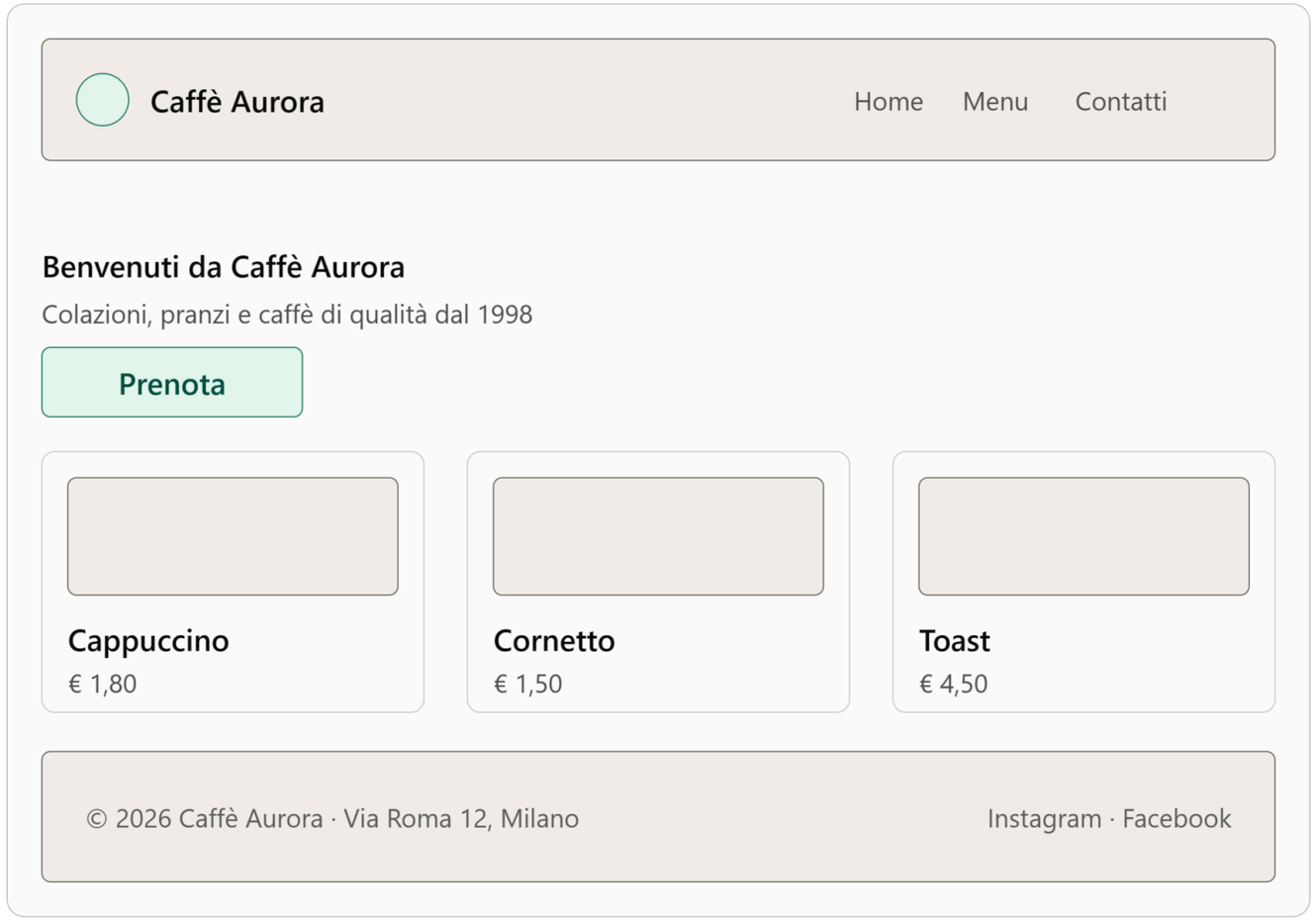
I componenti possono aiutarci a suddividere e dunque separare ordinatamente il codice riguardante il macro layout della nostra pagina.

Ad esempio, potremmo creare un componente per l'header, uno per il main, uno per una sidebar e uno per il footer!



Esempio Home Page

Come divideresti questa pagina in componenti?



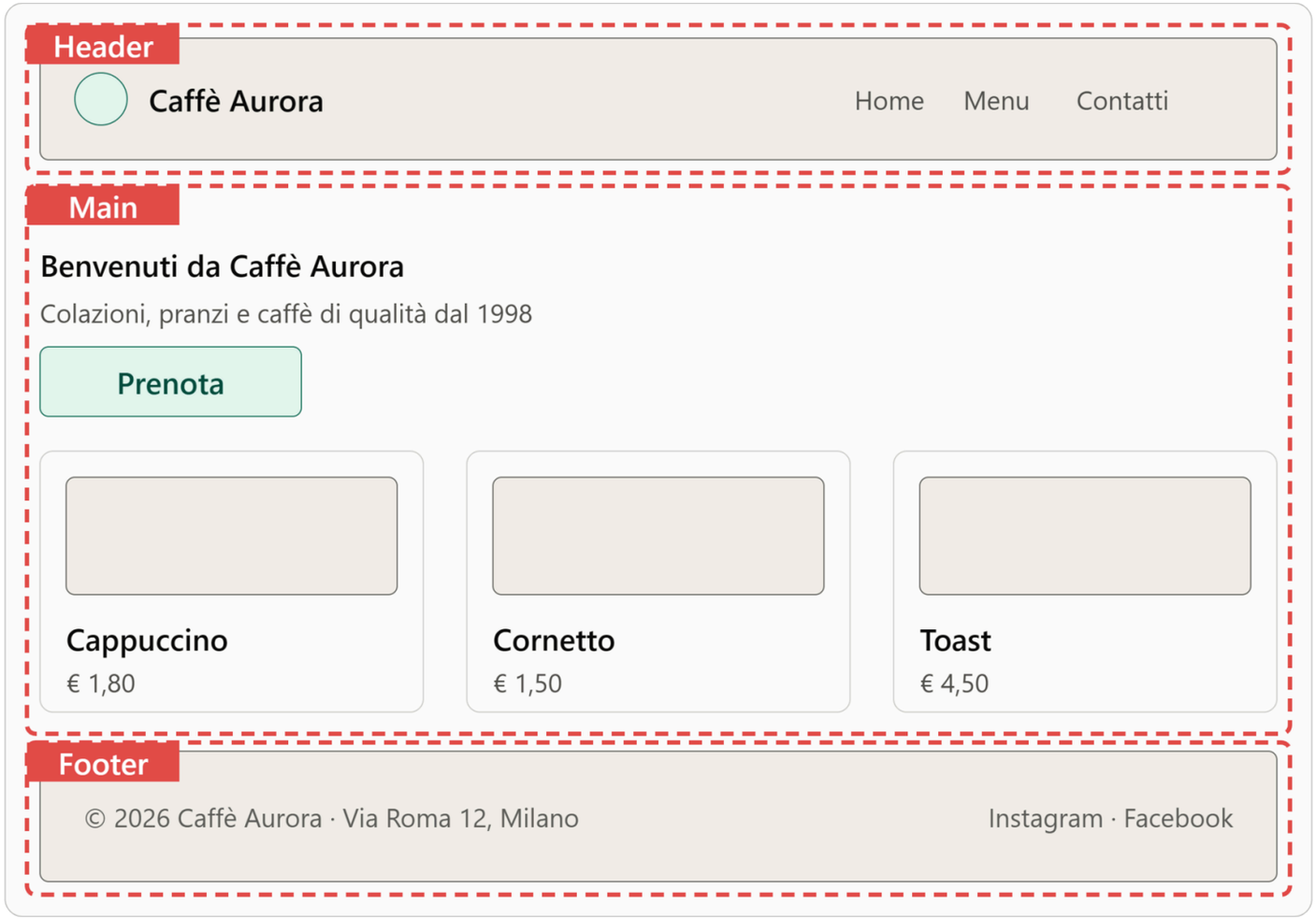
Esempio Home Page

Questo potrebbe essere un buon inzio!

<Header/>

<Main/>

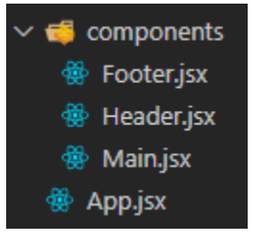
<Footer/>



Esempio Home Page

Creiamo 3 componenti e li utilizziamo in App.jsx!

```jsx
// Header.jsx
export default function Header() {
  return (<header> /* contenuto qui */ </header>);
}
```




```jsx
// Main.jsx
export default function Main() {
  return (<main> /* contenuto qui */ </main>);
}
```

```jsx
// Footer.jsx
export default function Footer() {
  return (<footer> /* contenuto qui */ </footer>);
}
```

```jsx
// App.jsx
import Header from "./components/Header.jsx";
import Main from "./components/Main.jsx";
import Footer from "./components/Footer.jsx";

export default function App() {
  return (
    <>
      <Header />
      <Main />
      <Footer />
    </>
  );
}
```

Componenti

Possiamo fare di più?

Ogni elemento della nostra pagina può diventare un componente.

Questo ci darà diversi vantaggi:

- Potremo riutilizzarlo senza scrivere ogni volta tutto il suo codice.
- Avremo un unico punto in cui andare ad apportare le nostre modifiche.
- Il codice del nostro componente avrà un file tutto suo e sarà quindi isolato dal resto, rendendo il codice più pulito e facile da mantenere.





LIVE CODING



JSX

Intro a JSX

Che cos'è JSX?

JavaScript XML

JSX è un'estensione della sintassi JavaScript che permette di scrivere la struttura dei vari componenti usando una sintassi **simile** al linguaggio **HTML**.

Possiamo facilmente **interpolare** espressioni e variabili JavaScript all'interno di JSX utilizzando le **parentesi graffe**.

```jsx
const title = "Titolo";

export default function BaseTitle() {
  return (<h1>{title}</h1>);
}
```



JSX

⚠️ JSX non è HTML!

Dobbiamo prestare **attenzione** quando utilizziamo attributi tipici dell'HTML all'interno del markup JSX.

Per esempio, **NON** possiamo utilizzare l'attributo '**class**' perché, in JavaScript, è una parola chiave riservata. In JSX, dobbiamo utilizzare '**className**' come alternativa.

Anche l'attributo **style** non si aspetta una stringa, ma un oggetto con una lista di proprietà CSS scritte in formato camelCase.

```
return (
    <div className="card" style={{ textAlign: "center" }}></div>
);
```

JSX

⚠️ JSX non è HTML! (parte 2)

Quando utilizziamo i componenti **self-closing** in JSX, dobbiamo obbligatoriamente inserire lo **slash di chiusura** all'interno del tag:

HTML

```html
<div>
  <br>
  <input type="text">
  <img src="..." alt="...">
</div>
```

JSX

```jsx
return (
  <div>
    <br />👈
    <input type="text" />👈
    <img src="..." alt="..." />👈
  </div>
);
```

JSX

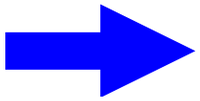
React Fragment: La soluzione alla 🖌️ "zuppa di "

Siccome all'interno di uno snippet di codice **JSX** può esistere solo un **root element**, React ci mette a disposizione un componente speciale, il <*React.Fragment*>.

```
// 🚫 Sintassi non consentita
return (
    <h1>titolo</h1>
    <h2>Sotto titolo</h2>
);
```

```
// 😎 Utilizzo del React.Fragment
return (
    <React.Fragment>
      <h1>titolo</h1>
      <h2>Sotto titolo</h2>
    </React.Fragment>
);
```

Esiste una **sintassi alternativa** e più utilizzata del React Fragment



```
return (
    <>
      <h1>titolo</h1>
      <h2>Sotto titolo</h2>
    </>
);
```



LIVE CODING